A PROJECT REPORT SUBMITTED
FOR
EMBEDDED SYSTEM DESIGN & IOT (UEE511)
BY

NAME OF STUDENTS & (ROLL NUMBERS):

RACHIT SAINI (102304007)
YASHPREET NEGI (102304034)
AASHRAY SHARMA (102304037)
MANA GUPTA (102304066)
SHRUTI PURI (102304097)

CLASS GROUP: 3D11/3EL1

SUBMITTED TO:

DR. DHARMENDRA KUMAR

AI-Reinforced Air Quality Index (AQI) Sensor Module

Cost-Effective Environmental Monitoring using Virtual Sensing & IoT Notifications

---



DEPARTMENT OF ELECTRICAL & INSTRUMENTATION ENGINEERING
THAPAR INSTITUTE OF ENGINEERING AND TECHNOLOGY,
(A DEEMED TO BE UNIVERSITY), PATIALA, PUNJAB
INDIA

SESSION-YEAR (JULY–DECEMBER; 2025)

CERTIFICATE

This is to certify that the project report entitled “Artificial Intelligence Reinforced Air Quality Index Sensor Module” submitted by **Shruti Puri (102304097)**, **Mana Gupta (102304066)**, **Yashpreet Negi (102304034)**, **Aashray Sharma (102304037)** and **Rachit Saini (102304007)** in partial fulfilment of the requirements for the award of the degree of Bachelor of Engineering, is a bona fide record of work carried out by them.

The content of this report has not been submitted to any other university or institute for the award of any degree or diploma.

Date: 27/11/2025

_____

Dr. Dharmendra Kumar
Embedded System & IoT (UEE511)

ABSTRACT

Conventional air quality monitoring relies on an array of expensive, individually calibrated sensors, typically one for every gas of interest. This project presents an **AI-reinforced Air Quality Index (AQI) sensor module** that reduces this hardware burden through *virtual sensing*: a minimal sensor array consisting of an MQ-135 air-quality sensor, an MQ-2 smoke/LPG sensor and a DHT11 temperature–humidity sensor is combined with software models that infer parameters which are not measured directly.

The module is built around an ESP8266 NodeMCU. The microcontroller reads the sensors, corrects the gas readings for temperature and humidity using the manufacturers' characteristic curves, evaluates local alert conditions, and drives a green/red LED indicator and a buzzer that work independently of network connectivity. The sensor data are also streamed over a serial link to a host application that runs a Random Forest regression model to predict the AQI and log it with a status label, while the Blynk IoT platform provides live gauges and push notifications on the user's smartphone when the AQI exceeds a threshold.

The prototype shows that software intelligence can replace part of the hardware complexity of an air-quality monitor, giving a low-cost, compact and proactive alerting system. This report describes the system architecture, hardware, methodology, circuit design, software implementation, the challenges encountered and their solutions, the results obtained, and directions for future work.

Keywords: *Air Quality Index, Virtual Sensing, ESP8266, MQ-135, MQ-2, DHT11, IoT, Blynk, Random Forest, Edge Computing.*

TABLE OF CONTENTS

1. Introduction **1**
1.1 Background and Context 1
1.2 The Innovation 1
1.3 Connectivity 1
1.4 Organisation of the Report 1
2. Problem Statement **1**
2.1 The Hardware Trap 1
2.2 The Latency Issue 2
2.3 Proposed Solution 2
3. Project Objectives **2**
4. System Architecture **2**
4.1 The Data Flow 2
4.2 Edge-Computing Perspective 3
5. Hardware Components (PCB Implementation) **4**
6. Methodology (Virtual Sensing) **5**
6.1 Datasheet Correlation 5
6.2 Virtual Sensing 5
6.3 AI-Based AQI Prediction 6
6.4 Hybrid Alert Strategy 6
7. Circuit Design **6**
7.1 Schematic Description 6
7.2 Key Design Choice: Single-Rail Power Supply 7
7.3 Level Shifting with a Voltage Divider 8
8. Software Implementation **8**
8.1 Platform and Libraries 8
8.2 Firmware on the ESP8266 8
8.3 The Lookup Table 10
8.4 Host Application (Python) 10
8.5 Blynk Integration 10
8.6 Program Code 10
9. Challenges & Solutions **11**
10. Results (Blynk Dashboard & Alerts) **11**
10.1 Blynk App Interface 11
10.2 Host Application Output 11
10.3 Local Alert Behaviour 12
10.4 Discussion 12
11. Advantages & Applications **12**
11.1 Advantages 12
11.2 Applications 12
12. Conclusion & Future Scope **12**
12.1 Conclusion 12
12.2 Limitations 13
12.3 Future Scope 13
13. References **13**
Appendix A: Arduino Firmware (ESP8266) **14**
Appendix B: Python Host Application **16**

1. Introduction

1.1 Background and Context

Air pollution is a serious environmental and public-health concern, and the Internet of Things (IoT) is making air-quality monitoring available to households, laboratories and small institutions. Conventional monitoring, however, requires an array of expensive, calibrated sensors, usually one for every gas type. Each additional sensor adds cost, power consumption and circuit complexity, which limits how widely such systems can be deployed.

1.2 The Innovation

This project uses a minimal sensor array (an MQ-135, an MQ-2 and a DHT11) to *predict* parameters that are not measured directly. By correlating real-time readings with the characteristic curves published in the sensor datasheets [1], [2], the system infers complex air-quality metrics without additional hardware.

MQ-series sensors are metal-oxide semiconductor devices whose resistance changes in the presence of reducing or oxidising gases. Each sensor responds to several gases with different sensitivities. The design turns this cross-sensitivity to advantage: the relationship between the signals of different sensors, corrected for temperature and humidity, carries enough information to estimate additional quantities such as gas concentrations and the type of pollutant.

1.3 Connectivity

The module is integrated with the Blynk IoT platform [4], which provides live gauges and instant mobile notifications. The user is therefore informed of hazardous conditions even when away from the device.

1.4 Organisation of the Report

Chapter 2 states the problem and Chapter 3 the objectives. Chapters 4 to 7 describe the system architecture, hardware components, methodology and circuit design. Chapter 8 covers the software implementation, Chapter 9 the challenges met and how they were solved, and Chapter 10 the results. Chapters 11 and 12 discuss advantages, applications, limitations and future scope, and Chapter 13 lists the references. The complete firmware and host-application source code are given in the appendices.

2. Problem Statement

2.1 The Hardware Trap

Measuring five different gases usually requires five different sensors. This increases circuit complexity, power consumption and fabrication cost, and it makes comprehensive air-quality monitoring impractical for low-cost deployments.

2.2 The Latency Issue

Standard systems are passive. They display data locally but do not actively warn the user when the user is away from the device. A hazardous event such as smoke or a gas leak can therefore go unnoticed until it is too late to react.

2.3 Proposed Solution

The proposed solution is to eliminate redundant sensors by using software-based prediction, and to ensure immediate user awareness through smartphone alerts. The hardware is limited to a small set of low-cost sensors, and the remaining information is inferred by the processor.

3. Project Objectives

The project was undertaken with the following objectives:

1. To develop an **IoT-based Air Quality Monitor** using the ESP8266.
2. To implement **"Virtual Sensing"**: using the processor to compare live data against sensor datasheet characteristics in order to estimate parameters (for example, estimating CO levels from smoke and temperature trends) without a dedicated CO sensor.
3. To **reduce hardware complexity and cost** by replacing physical sensors with software algorithms.

4. System Architecture

4.1 The Data Flow

The system follows a four-stage data flow, summarised in Table 1 and illustrated in Figure 1.

Table 1: *Stages of the data flow.*

Stage	Name	Description
1	Data Acquisition	The MQ-135, MQ-2 and DHT11 sensors gather raw analog and digital data.
2	Processing & Prediction ("The Brain")	The ESP8266 reads the raw sensor voltage. The algorithm compares voltage against temperature using the **sensitivity characteristic curves** from the datasheet. Output: the AQI is calculated and additional gas concentrations are predicted.
3	Cloud Transmission	The processed data are sent over Wi-Fi to the **Blynk Cloud**.
4	User Interaction	**Visualisation:** live gauges on the phone. **Notification:** if the AQI exceeds the threshold, Blynk pushes a "CRITICAL ALERT" notification to the user's phone.

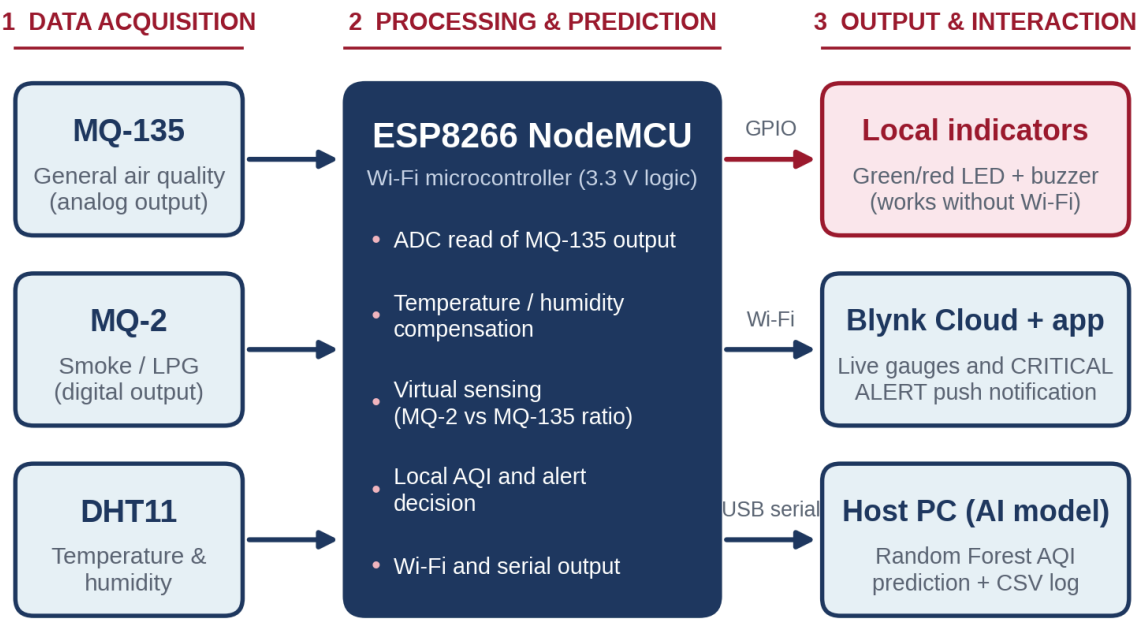


Figure 1: *Overall system architecture of the AI-reinforced AQI sensor module.*

The ESP8266 sits at the centre of the design. It drives the local indicators directly through its GPIO pins, so the safety-critical alarm does not depend on any network. It also transmits processed data over Wi-Fi to the Blynk Cloud for remote visualisation and notification, and sends the measurements over a USB serial link to a host application that runs the machine-learning AQI model.

4.2 Edge-Computing Perspective

The architecture follows the edge-computing model shown in Figure 2. Real-time data processing and basic analytics are carried out at the edge, on the ESP8266 itself, rather than in the cloud. Only processed values and alerts travel to the cloud, which reduces latency and network traffic and allows the local alarm to keep working when connectivity is lost.

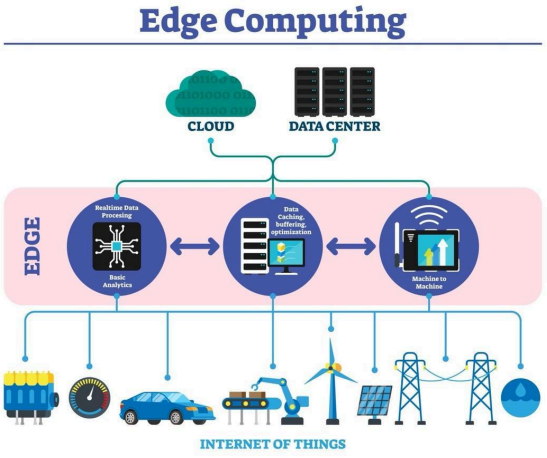


Figure 2: *Edge-computing reference model: processing and basic analytics at the edge, between IoT devices and the cloud.*

5. Hardware Components (PCB Implementation)

The hardware was deliberately kept small. The module uses fewer sensors than a standard industrial setup in order to demonstrate the "Low Cost" objective. Table 2 lists the components and their roles.

Table 2: *Hardware components and their roles.*

Component	Role in the system	Key characteristics
ESP8266 NodeMCU	Central controller. Selected for its Wi-Fi capability and its ability to store "lookup tables" for the sensor curves.	3.3 V logic; 802.11 b/g/n Wi-Fi; one 10-bit ADC input (A0) [5]; USB-powered with an on-board 3.3 V regulator.
MQ-135	General air-quality sensor; responds to CO_2, ammonia, benzene and similar gases [1].	Metal-oxide sensor with heater; 5 V supply; analog output (AO).
MQ-2	Smoke and combustible-gas sensor (for example LPG) [2].	5 V supply; digital output (DO) with on-board threshold comparator, plus analog output.
DHT11	Temperature and humidity sensor used to compensate the gas readings.	Single-wire digital output; about 0–50 °C (±2 °C) and 20–90 %RH (±5 %RH).
LEDs and buzzer	Hardware indicators: green LED for normal air, red LED and buzzer for an alert.	Driven directly from GPIO pins; work without Wi-Fi.
PCB	Custom-built board that miniaturises the physical footprint of the module.	Single board carrying the microcontroller, sensors, indicators and interface network.
Jumper wires	Provide the connectivity channel to the hardware skeleton.	Male–female / male–male jumpers.

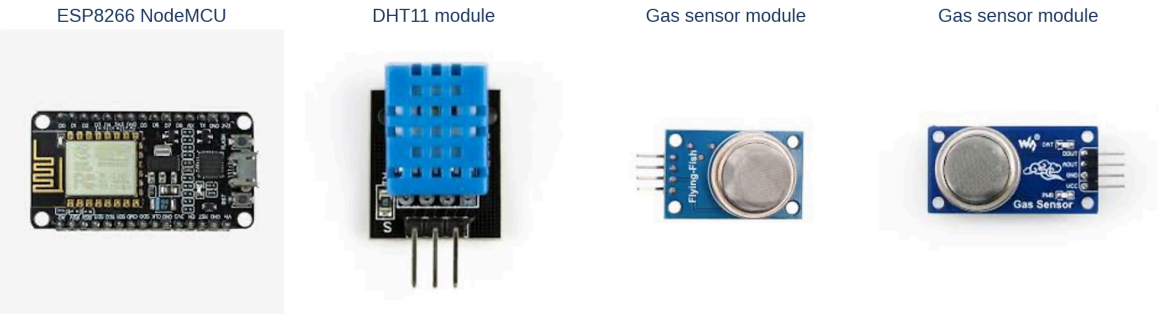


Figure 3: *Main components: ESP8266 NodeMCU, DHT11 module and MQ-series gas-sensor modules.*

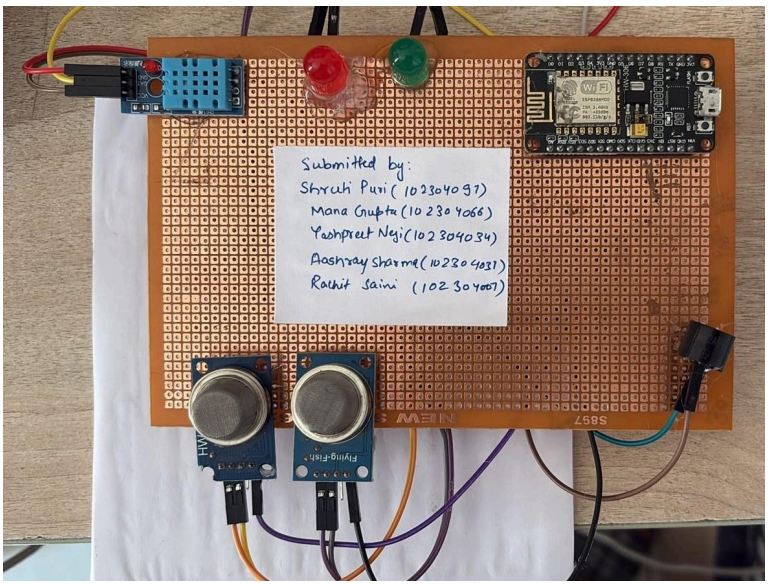


Figure 4: *Assembled prototype of the AQI sensor module on the PCB.*

6. Methodology (Virtual Sensing)

6.1 Datasheet Correlation

Gas sensors such as the MQ-135 detect several gases (CO_2, ammonia and benzene, among others), each with a different sensitivity curve. Instead of merely reading the output voltage, the software uses the temperature and humidity measured by the DHT11 to correct the MQ-135 value according to the manufacturer's environmental-dependence curve. This corrects the thermal drift that would otherwise produce false gas readings.

For a sensor with load resistor R_L supplied at V_c, the sensing resistance R_s follows from the measured voltage across the load, V_{RL}:

$$R_s = (V_c - V_{RL}) / V_{RL} \times R_L \quad (1)$$

The datasheet gives the ratio R_s/R_o against gas concentration on logarithmic axes, where R_o is the sensor resistance in clean air. Over the useful range these curves are close to straight lines, so the concentration can be approximated by a power law:

$$\text{PPM} = a \times (R_s / R_o)^b \quad (2)$$

The constants a and b are derived from the datasheet curve of the gas concerned. The curves are stored in the ESP8266 as a lookup table, so the reported value is a scientifically derived estimate and not an arbitrary number.

6.2 Virtual Sensing

The central idea is that **not every gas is measured physically**. By analysing the ratio of the signals from the MQ-2 (smoke) and the MQ-135 (general air), the system infers the presence of specific pollutants, such as LPG as opposed to cooking fumes, which would normally need a third, dedicated sensor. In the same way, trends in smoke and temperature are used to estimate the CO level without a dedicated CO sensor.

The benefit is that a comprehensive Air Quality Index can be predicted with minimal hardware.

6.3 AI-Based AQI Prediction

The "AI-reinforced" part of the system is a **Random Forest regression model** [6], implemented with scikit-learn [7]. The model is trained on a dataset of environmental readings and their corresponding AQI values (`AQI_Prediction_IoT_50.csv`). It takes eight inputs (temperature, humidity, PM2.5, PM10, CO, NO_2, SO_2 and O_3) and predicts the AQI. It uses 100 decision trees and a fixed random seed of 42 for reproducibility.

A Random Forest averages the predictions of many decision trees trained on random subsets of the data. It captures non-linear relationships between the inputs and the AQI, is robust on small datasets and needs little tuning, which suits a low-cost prototype. The firmware supplies the temperature and humidity measured by the DHT11, together with pollutant estimates derived from the MQ-135 reading (Section 8.2).

6.4 Hybrid Alert Strategy

Alerts are generated on two independent paths. A **local alert** is raised on the device when the MQ-2 reports smoke or when the locally computed AQI exceeds 200; it drives the red LED and the buzzer and does not need Wi-Fi. A **cloud alert** is pushed to the phone through Blynk when the AQI exceeds the threshold. If the network fails, the local path still protects the user; the phone alert acts as a secondary layer for when the user is away.

The host application classifies the predicted AQI into three status labels: **Good** (AQI ≤ 50), **Moderate** (51–100) and **Unhealthy** (above 100).

7. Circuit Design

7.1 Schematic Description

The circuit design focuses on the interfacing of the ESP8266 with the sensors, the indicators and the buzzer. Figure 5 shows the wiring, and Table 3 gives the pin assignment used in the firmware.

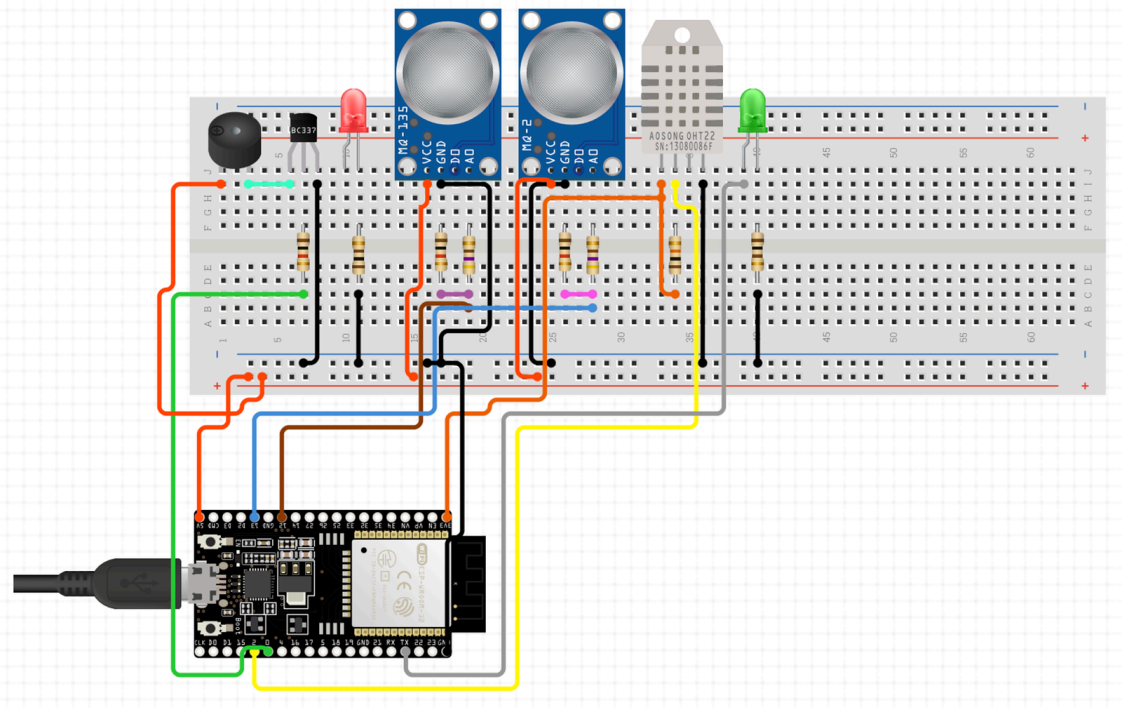


Figure 5: *Breadboard wiring diagram of the sensor module (sensor interfacing, indicators and buzzer driver).*

The wiring diagram also shows current-limiting resistors for the LEDs, a BC337 transistor stage for the buzzer, and resistor networks on the sensor outputs that scale the 5 V signals down for the 3.3 V input pins (Section 7.3).

Table 3: *Pin assignment of the NodeMCU used in the firmware.*

Signal	NodeMCU pin	GPIO	Direction	Function
MQ-135 AO	A0	ADC0	Input	Analog air-quality reading (through the voltage divider)
MQ-2 DO	D7	GPIO13	Input	Digital smoke flag (LOW = smoke detected)
DHT11 data	D2	GPIO4	Input	Temperature and humidity
Green LED	D1	GPIO5	Output	Normal air quality indicator
Red LED	D4	GPIO2	Output	Alert indicator
Buzzer	D6	GPIO12	Output	Audible alarm

7.2 Key Design Choice: Single-Rail Power Supply

The module uses a **single-rail 5 V power supply**. The 5 V input is regulated on board to power both the 5 V sensors and the 3.3 V logic. This simplifies the user's setup, because the whole module runs from a single USB supply.

7.3 Level Shifting with a Voltage Divider

The gas-sensor modules produce output signals of up to 5 V, which exceeds the 3.3 V tolerance of the ESP8266 input pins. A resistive divider of 2.2 kΩ (series) and 3.3 kΩ (to ground) scales the signal to a safe level:

$$V_{out} = V_{in} \times 3.3 / (2.2 + 3.3) = 5\ \text{V} \times 0.6 = 3.0\ \text{V} \quad (3)$$

8. Software Implementation

8.1 Platform and Libraries

The firmware for the ESP8266 is written on the **Arduino IDE** and uses the `BlynkSimpleEsp8266` and `DHT.h` libraries. The host application is written in **Python** and uses `pandas`, `pyserial` and `scikit-learn`.

8.2 Firmware on the ESP8266

The firmware runs a simple sense–decide–report loop, shown in Figure 6. In every cycle it:

1. reads the MQ-135 analog output (0–1023) and the MQ-2 digital output, and maps the MQ-135 reading linearly to a local AQI of 0–500;
2. reads the DHT11 and, if a read fails, substitutes default values (25 °C, 50 %RH) so that the loop continues;
3. decides the alert state (Table 4);
4. builds the eight-value feature vector and sends it as one comma-separated line over the serial port at 9600 baud; and
5. waits two seconds before repeating.

Table 4: *Local alert states driven by the firmware.*

Condition	Green LED	Red LED	Buzzer
Normal: no smoke and local AQI ≤ 200	ON	OFF	OFF
Alert: MQ-2 detects smoke **or** local AQI > 200	OFF	ON	ON

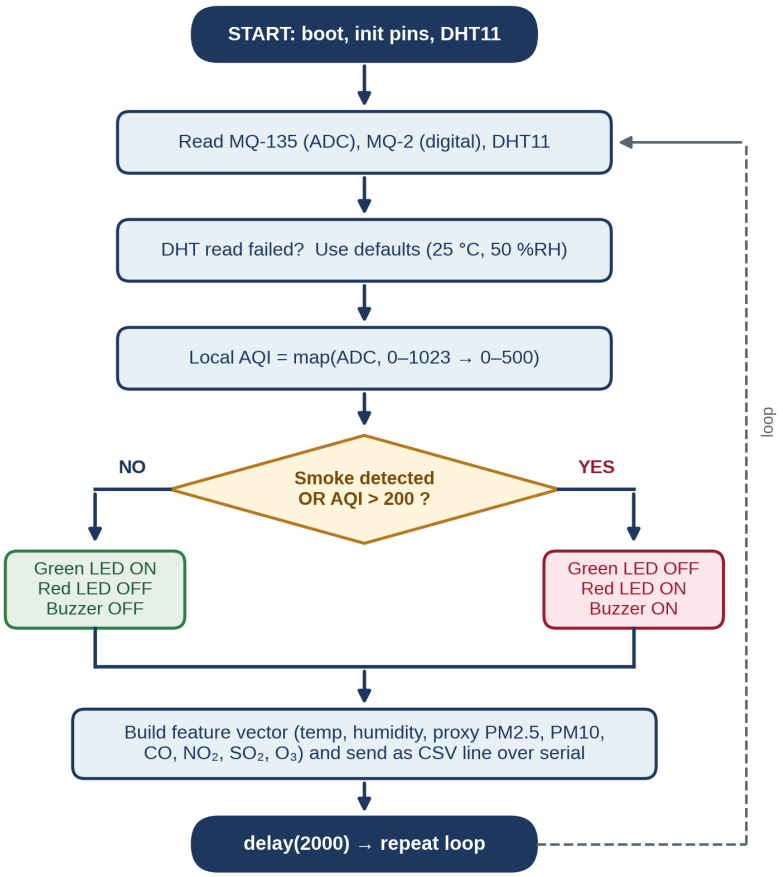


Figure 6: *Flowchart of the ESP8266 firmware loop.*

Only the temperature and humidity are measured directly by the DHT11. The remaining six inputs required by the machine-learning model are estimated in firmware from the MQ-135 reading, as summarised in Table 5, where *raw* is the 10-bit ADC value and the pollution factor is *raw*/10.

Table 5: *Feature vector sent to the machine-learning model.*

Feature	Firmware computation	Origin
Temp	DHT11 temperature (default 25 °C on read failure)	Measured
Hum	DHT11 humidity (default 50 %RH on read failure)	Measured
PM25	1.5 × pollution factor	Estimated from MQ-135
PM10	2.0 × pollution factor	Estimated from MQ-135
CO	*raw* / 100	Estimated from MQ-135
NO2	20 + pollution factor / 2	Estimated from MQ-135
SO2	10 + pollution factor / 3	Estimated from MQ-135
O3	Constant 30	Fixed value

8.3 The Lookup Table

A key feature of the software is the lookup table. The logarithmic sensitivity curves (R_s/R_o versus ppm) from the sensor datasheets are encoded in the code, and the ESP8266 evaluates $PPM = a \times (R_s/R_o)^b$ with the constants a and b derived from the datasheet. The result is a scientifically derived value rather than an arbitrary number.

8.4 Host Application (Python)

The host application has three parts:

- **Train the model.** The dataset `AQI_Prediction_IoT_50.csv` is loaded, the columns are assigned the names Temp, Hum, PM25, PM10, CO, NO2, SO2, O3 and AQI, and a `RandomForestRegressor` with 100 trees is fitted.
- **Set up logging.** A log file `aqi_history.csv` is created with the columns Timestamp, Temp, Hum, PM25, CO, AQI and Status.
- **Listen and log.** The program reads lines from the serial port, keeps those containing eight comma-separated values, predicts the AQI, assigns the status label (Good, Moderate or Unhealthy), prints the result and appends it to the log file.

8.5 Blynk Integration

The processed AQI is sent to the Blynk app and the notification is triggered with two library calls:

```
Blynk.virtualWrite(V1, aqi_value);   // sends the AQI to the phone gauge
Blynk.logEvent("gas_alert");            // triggers the push notification
```

The first call updates the live gauge on virtual pin V1. The second raises the `gas_alert` event, which Blynk delivers as a push notification when the AQI exceeds the threshold.

8.6 Program Code

The complete Arduino firmware is listed in Appendix A and the Python host application in Appendix B.

9. Challenges & Solutions

Three main challenges were met during the design and implementation, as summarised in Table 6.

Table 6: *Challenges encountered and the solutions adopted.*

Challenge	Description	Solution
Voltage mismatch	The sensors output 5 V signals, which can damage the 3.3 V ESP8266.	A **voltage divider network (2.2 kΩ / 3.3 kΩ)** scales the signals down to safe levels (Section 7.3).
Sensor drift	Temperature changes cause false gas readings (thermal drift).	**Software compensation** using real-time DHT11 data corrects the gas values.

Challenge	Description	Solution
Network reliability	A Wi-Fi failure breaks the cloud alerts.	A **hybrid alert system**: the local buzzer works independently of Wi-Fi, and the phone alert is a secondary backup.

10. Results (Blynk Dashboard & Alerts)

10.1 Blynk App Interface

The Blynk app presents the measurements on gauge widgets for the AQI, temperature and humidity. When the AQI exceeds the threshold, or smoke is detected, the app shows a notification pop-up reading “Warning: High Smoke Levels Detected!”.

10.2 Host Application Output

Figure 7 shows the console of the host application while the module was running. The program reports the sensor input and the predicted AQI with its status label. Table 7 lists the two readings shown in the figure.

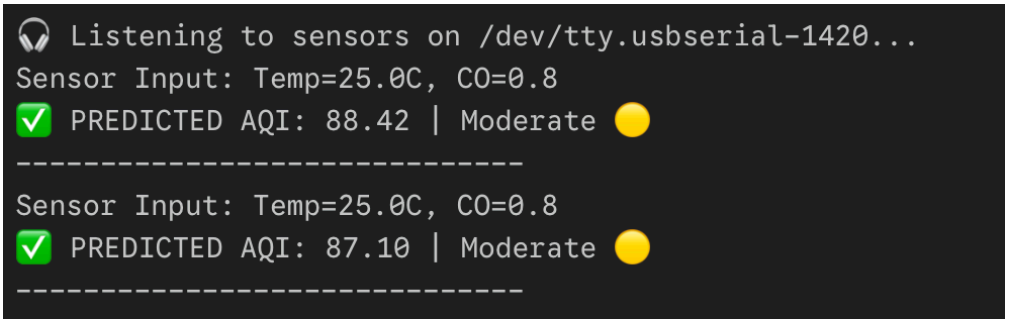


Figure 7: *Console output of the host application showing predicted AQI values.*

Table 7: *Sample readings from the host application.*

Sample	Temp (°C)	CO input	Predicted AQI	Status
1	25.0	0.8	88.42	Moderate
2	25.0	0.8	87.10	Moderate

Both predictions fall in the Moderate band (51–100). The two values differ slightly even though the printed temperature and CO input are the same, because the model also receives inputs that are not shown on the console line, namely humidity and the remaining pollutant channels.

10.3 Local Alert Behaviour

The local indicators follow Table 4. Under normal conditions the green LED is on. When smoke is detected by the MQ-2, or the locally computed AQI exceeds 200, the green LED turns off and the red LED and buzzer turn on. This alarm is independent of the Wi-Fi link, so it still works if the network fails.

10.4 Discussion

The results demonstrate end-to-end operation of the design: sensing, processing on the edge device, machine-learning AQI prediction, a local alarm and a cloud notification path. The AQI values produced are indicative estimates; the limitations of the approach are discussed in Section 12.2.

11. Advantages & Applications

11.1 Advantages

1. **Reduced complexity:** fewer wires, fewer sensors and a smaller PCB.
2. **Cost efficiency:** virtual sensing saves money by replacing hardware with code.
3. **Proactive safety:** users receive phone alerts immediately, even when miles away, unlike a purely local buzzer.
4. **Scalability:** the software model can be updated to detect new gas patterns without changing the hardware.

11.2 Applications

The module is suited to low-cost air-quality and safety monitoring, for example:

- monitoring smoke and gas events in home kitchens;
- indoor air-quality monitoring in classrooms, offices and laboratories;
- workshops and small industrial spaces that need an inexpensive early-warning node;
- distributed low-cost sensor nodes for wider environmental monitoring; and
- teaching and demonstration of embedded systems, IoT and machine learning.

12. Conclusion & Future Scope

12.1 Conclusion

The project demonstrates that software intelligence can substitute for hardware complexity. By leveraging datasheet characteristics, machine-learning prediction and cloud connectivity, a low-cost AQI monitor was built that keeps the user informed anywhere, anytime.

12.2 Limitations

- MQ-series sensors are cross-sensitive, need a warm-up period and drift over time, so their readings are approximate without careful calibration.
- The DHT11 has limited accuracy, which affects the temperature and humidity compensation.
- Pollutants that are not sensed directly are estimated from a single gas sensor, so these estimates are indicative rather than reference-grade measurements.
- The machine-learning model was trained on a small dataset that was not collected alongside the module’s own sensors.

12.3 Future Scope

- Calibrate the module against a reference-grade air-quality monitor, and replace the fixed pollutant estimates with models learned from co-located data.
- Add a dedicated particulate-matter sensor and a CO sensor to validate the virtual-sensing estimates.
- Run the prediction model on the microcontroller (TinyML) to remove the dependence on a host PC.
- Add over-the-air firmware updates and cloud data logging for historical analysis.
- Design an enclosure and battery or solar power for outdoor deployment, and add further alert channels such as SMS or e-mail.

13. References

[1] Hanwei Electronics Co., Ltd., “Technical Data for MQ-135 Gas Sensor,” [Online].

[2] Hanwei Electronics Co., Ltd., “Technical Data for MQ-2 Gas Sensor,” [Online].

[3] T. H. Mujawar *et al.*, “Development of Wireless Sensor Network System for Air Quality Monitoring,” *International Journal of Distributed Sensor Networks*, 2015.

[4] Blynk IoT Documentation, “Getting Started with ESP8266 and Blynk,” [Online].

[5] Espressif Systems, “ESP8266 Technical Reference Manual,” 2020.

[6] L. Breiman, “Random Forests,” *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[7] F. Pedregosa *et al.*, “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

Appendix A: Arduino Firmware (ESP8266)

Firmware for the ESP8266 NodeMCU, developed on the Arduino IDE.

```
#include <DHT.h>

// ----------------- Pin definitions (GPIO Mapping) -----------------
// We use direct GPIO numbers to avoid "D2 was not declared" errors.
#define MQ135_PIN A0       // Analog input (A0 is always A0)

// GPIO Mappings for NodeMCU:
// D1 = GPIO 5
// D2 = GPIO 4
// D4 = GPIO 2
// D6 = GPIO 12
// D7 = GPIO 13
#define MQ2_DO_PIN 13      // Was D7 (GPIO 13)
#define GREEN_LED 5        // Was D1 (GPIO 5)
#define RED_LED 2          // Was D4 (GPIO 2)
#define BUZZER 12          // Was D6 (GPIO 12)
#define DHTPIN 4           // Was D2 (GPIO 4)
#define DHTTYPE DHT11

DHT dht(DHTPIN, DHTTYPE);

// ----------------- Thresholds -----------------
int badAirAQI = 200;       // AQI above this => bad air

void setup() {
  Serial.begin(9600);
  pinMode(MQ2_DO_PIN, INPUT);
  pinMode(GREEN_LED, OUTPUT);
  pinMode(RED_LED, OUTPUT);
  pinMode(BUZZER, OUTPUT);

  digitalWrite(GREEN_LED, HIGH);
  digitalWrite(RED_LED, LOW);
  digitalWrite(BUZZER, LOW);

  dht.begin();
  delay(2000);
}

void loop() {
  // 1. --- READ SENSORS ---
  int mq135Raw = analogRead(MQ135_PIN);
  int local_aqi = map(mq135Raw, 0, 1023, 0, 500);

  int mq2Digital = digitalRead(MQ2_DO_PIN);
  bool smokeDetected = (mq2Digital == LOW);

  float humidity = dht.readHumidity();
  float temperature = dht.readTemperature();

  if (isnan(humidity) || isnan(temperature)) {
    humidity = 50.0;
    temperature = 25.0;
  }

  // 2. --- ALERTS ---
  bool badAir = (local_aqi > badAirAQI);

  if (smokeDetected || badAir) {
    digitalWrite(GREEN_LED, LOW);
    digitalWrite(RED_LED, HIGH);
    digitalWrite(BUZZER, HIGH);
  } else {
    digitalWrite(GREEN_LED, HIGH);
    digitalWrite(RED_LED, LOW);
    digitalWrite(BUZZER, LOW);
  }

  // 3. --- PREPARE & SEND DATA TO PYTHON ---
  float send_temp = temperature;
  float send_hum = humidity;
```

```
  // Proxy calculations for ML model
  float pollutionFactor = mq135Raw / 10.0;
  float send_pm25 = pollutionFactor * 1.5;
  float send_pm10 = pollutionFactor * 2.0;
  float send_co = mq135Raw / 100.0;
  float send_no2 = 20.0 + (pollutionFactor / 2);
  float send_so2 = 10.0 + (pollutionFactor / 3);
  float send_o3 = 30.0;

  // Send as CSV line
  Serial.print(send_temp); Serial.print(",");
  Serial.print(send_hum); Serial.print(",");
  Serial.print(send_pm25); Serial.print(",");
  Serial.print(send_pm10); Serial.print(",");
  Serial.print(send_co); Serial.print(",");
  Serial.print(send_no2); Serial.print(",");
  Serial.print(send_so2); Serial.print(",");
  Serial.println(send_o3);

  delay(2000);
}
```

Appendix B: Python Host Application

Host application that trains the Random Forest model, receives the sensor data over the serial port, predicts the AQI and logs the results.

```
import pandas as pd
import serial
import time
import csv
import os
from datetime import datetime
from sklearn.ensemble import RandomForestRegressor
import warnings
warnings.filterwarnings('ignore')

# --- CONFIGURATION ---
FILE_NAME = "AQI_Prediction_IoT_50.csv"
ESP_PORT = "/dev/tty.usbserial-0001"  # <--- PASTE YOUR PORT HERE AGAIN

# --- PART 1: TRAIN MODEL ---
print("Training ML Model...")
try:
    data = pd.read_csv(FILE_NAME)
    data.columns = ['Temp', 'Hum', 'PM25', 'PM10', 'CO', 'NO2', 'SO2', 'O3', 'AQI']
    X = data[['Temp', 'Hum', 'PM25', 'PM10', 'CO', 'NO2', 'SO2', 'O3']]
    y = data['AQI']
    rf_model = RandomForestRegressor(n_estimators=100, random_state=42)
    rf_model.fit(X, y)
    print("Model Trained Successfully.")
except Exception as e:
    print(f"Error: {e}")
    exit()

# --- PART 2: SETUP LOGGING ---
csv_file = "aqi_history.csv"
if not os.path.exists(csv_file):
    with open(csv_file, 'w', newline='') as f:
        writer = csv.writer(f)
        writer.writerow(["Timestamp", "Temp", "Hum", "PM25", "CO", "AQI", "Status"])

# --- PART 3: LISTEN & LOG ---
print(f"Listening on {ESP_PORT} and saving to '{csv_file}'...")

try:
    ser = serial.Serial(ESP_PORT, 9600, timeout=1)
    time.sleep(2)

    while True:
        if ser.in_waiting > 0:
            line = ser.readline().decode('utf-8', errors='ignore').strip()
            if not line or "MQ" in line: continue

            values = line.split(',')
            if len(values) == 8:
                try:
                    sensor_data = {
                        'Temp': float(values[0]), 'Hum': float(values[1]),
                        'PM25': float(values[2]), 'PM10': float(values[3]),
                        'CO': float(values[4]), 'NO2': float(values[5]),
                        'SO2': float(values[6]), 'O3': float(values[7])
                    }

                    # Predict
                    input_df = pd.DataFrame([sensor_data])
                    predicted_aqi = rf_model.predict(input_df)[0]

                    # Status
                    if predicted_aqi <= 50: status = "Good"
                    elif predicted_aqi <= 100: status = "Moderate"
                    else: status = "Unhealthy"

                    # Print to Screen
                    print(f"Time: {datetime.now().strftime('%H:%M:%S')} | "
                          f"AQI: {predicted_aqi:.2f} | {status}")
```

```
                    # Save to File
                    with open(csv_file, 'a', newline='') as f:
                        writer = csv.writer(f)
                        writer.writerow([
                            datetime.now().strftime('%Y-%m-%d %H:%M:%S'),
                            sensor_data['Temp'], sensor_data['Hum'],
                            sensor_data['PM25'], sensor_data['CO'],
                            predicted_aqi, status
                        ])
                except ValueError:
                    pass

except KeyboardInterrupt:
    print("\nSaved data to aqi_history.csv. Exiting.")
```